

Cloudflare Security Audit Playbook

Install Cloudflare's free security audit skill in Claude Code or Codex, run your first audit, and understand the report it writes.

Why I bothered with this one

Cloudflare open sourced the skill that seeded its internal vulnerability harness. That harness turned roughly 20,799 raw candidates into 7,245 actionable findings across 145 of Cloudflare's own repositories, and the write-up went live on 18 June 2026. The public skill is the single-repo starting point that whole system grew out of, it is MIT licensed, and it had passed 26,000 GitHub stars by October 2026.

The reason I care is simpler than the numbers. Most of what I ship was vibe coded. An agent wrote the auth, an agent wrote the database calls, and nobody has ever read that code with bad intentions. Asking an agent "is my app secure?" gets you a confident paragraph and a list of things that are not actually wrong. It reads your favourite three files, finds nothing, and tells you it looks fine.

This skill is the opposite of that. It forces the agent to write down what it is accountable for, then it sends a second agent to try to tear down everything the first one claimed. What survives is short. Short is what you want, because a report with four real bugs gets fixed and a report with 400 maybes gets closed.

01

How it actually works

The audit runs in six phases, and every phase reads the file the one before it wrote. Here is the shape of it.

1. Reconnaissance and hunting. The agent maps your architecture, trust boundaries and input surfaces into `architecture.md`, and writes a `coverage-ledger.json` listing the units it has to cover. Isolated hunters then take units off that ledger and record what they checked, while coverage critics look for the gaps nobody touched.
2. Validation. Every unique candidate goes to a fresh verifier whose only job is to disprove it. The agent that found the bug is never the agent that signs off on it. Results land in `findings.json` under three verdicts: `confirmed` needs a complete source trace and a bounded observed result, `needs_validation` records the exact unresolved fact and gets no severity at all, and `rejected` keeps the disproved candidate on file so a later run does not hand it back to you.
3. Verification and reporting. Fresh agents re-check the source claims behind the final records, and any material replacement gets another independent verifier. Only then does the skill derive `REPORT.md`, `FINDINGS-DETAIL.md` and `NEEDS-VALIDATION.md` from the verified records. Two zero-dependency Node validators, `validate-coverage-ledger.cjs` and `validate-findings.cjs`, run throughout to check the agent's own output against `report-schema.json`.

The one thing almost everyone gets wrong

The skill expects an OS-enforced sandbox before it will execute anything the target controls: builds, tests, processes, browsers, emulators, fuzzers, fixtures. The sandbox has to disable external networking, use a sanitized allowlisted environment, enforce resource limits, and only allow writes to assigned scratch paths. Without those controls the workflow does not run the suspicious code, it parks the lead as `needs_validation` instead.

WATCH OUT

If your first report comes back thin and `NEEDS-VALIDATION.md` is longer than `REPORT.md`, nothing is broken. Your agent could not safely execute the target code, so it refused to confirm anything it could not observe. Read `NEEDS-VALIDATION.md` anyway, because each entry names the exact fact that blocked it, and some of those you can resolve by hand in two minutes.

The second thing people miss: one pass is not the audit. Cloudflare says plainly in the repo that in their test runs, a single run found roughly half the vulnerabilities that repeated runs found in total. Runs against the same repo are additive, because the skill reads the previous ledger and findings to target gaps, revalidate changed source, and carry forward evidence that still matches the current code. Budget for two or three runs, not one.

The files

Two sets of files matter: the ones you install, and the ones the audit writes. Neither is a black box, and reading them is how you learn what the agent is actually doing.

SOURCE CODE

The full repo, MIT licensed:

<https://github.com/cloudflare/security-audit-skill>

WHAT YOU INSTALL (THE SKILL FOLDER)

SKILL.md	setup, core principles, workflow overview, anti-patterns
RECONNAISSANCE.md	phase 1 prompts and synthesis
HUNTING.md	phase 2 orchestration and validation rules
VALIDATION-AND-REPORTING.md	phases 3-6
ATTACK-CLASSES.md	core, wildcard and obvious-things prompts
AI-AND-LLM.md	prompt injection, agent/tool abuse, output handling
WEB-PROTOCOL-AND-AUTH.md	HTTP request framing, cache, auth protocols
CLIENT-SIDE.md	DOM injection, messaging trust, UI redress, prototype pollution
SUPPLY-CHAIN-AND-RELEASE.md	dependencies, CI, release, signing, updates, plugins
CLOUD-AND-DEPLOYMENT.md	IAM, IaC, containers, serverless, ingress, runtime config
DATA-ISOLATION-AND-LIFECYCLE.md	tenant isolation, cache, export, backup, deletion, restore
PROTOCOLS-RPC-AND-MESSAGING.md	RPC, serialization, queues, brokers, webhooks, streaming
RESOURCE-EXHAUSTION-AND-AVAILABILITY.md	shared resources, quotas, workers, operator spend
DESKTOP-MOBILE-AND-LOCAL-IPC.md	native apps, deep links, webviews, daemons, local IPC
MEMORY-SAFETY-AND-BINARY.md	memory safety, binary and kernel classes
report-schema.json	schema for all three findings.json verdicts
validate-findings.cjs	zero-dependency findings validator
validate-coverage-ledger.cjs	zero-dependency ledger validator
validate-findings.test.cjs	validator tests and fixture checks
validate-coverage-ledger.test.cjs	ledger validator tests

WHAT THE AUDIT WRITES (YOUR OUTPUT DIRECTORY)

architecture.md	phase 1: architecture, trust boundaries, input surfaces
coverage-ledger.json	phase 1-5: the units to cover and what was checked
findings.json	phase 4-5: confirmed / needs_validation / rejected records
REPORT.md	phase 6: the summary you read first
FINDINGS-DETAIL.md	phase 6: full write-up per finding, with the fix
NEEDS-VALIDATION.md	phase 6: leads blocked by an unresolved fact

If you only open one of those, open **REPORT.md**. If you only open two, the second is **NEEDS-VALIDATION.md**, because that is where the bugs the agent could not prove are sitting, and some of them are real.

Set it up in Claude Code or Codex

Install takes about a minute. The skill works with any coding agent whose model supports tool use and parallel sub-agents, which covers Claude Code and OpenAI Codex.

1. Check Node.js is installed. Run `node --version` in your terminal. The two validators are plain Node with zero dependencies, and without Node the agent cannot check its own findings against the schema.
2. Open a terminal in the project you want audited. The skill installs per project by default, which is what I want when I am trying it for the first time.
3. Install it with the Skills CLI, using the install command in the next section. The CLI pulls the repo and writes the skill folder for you, so there is no manual clone step and no copying files around.
4. If you would rather have it everywhere, add `--global` to that same command for a user-level install. I run global, because I never remember to install it in the repo I am worried about.
5. Check where it landed. For Claude Code a project install sits in the project's skills folder and a global install sits under `~/.claude/skills/security-audit/`. For Codex it goes in the project folder, or `.codex/skills/security-audit/` with a pointer line in `AGENTS.md` so the agent knows to read it.
6. The folder name has to match the `name:` field at the top of `SKILL.md`. The CLI gets this right on its own. It only bites you if you moved or renamed the folder by hand afterwards, and the symptom is the skill never triggering.
7. Decide where reports go. If you do not name a directory, full audit mode writes to `~/security-audit-skill/<repo-name>/run-<N>`. The workflow only writes inside your repo when you explicitly pick a path that version control ignores, which is the behaviour you want.
8. Run your first audit. Start your agent inside the repo and ask for it in plain language, for example `security audit this codebase`. The skill matches on the request itself, so there is no slash command to memorise.
9. Watch for the first real sign it worked: the agent writes `architecture.md` and `coverage-ledger.json` before it starts hunting. If it jumps straight to listing vulnerabilities, the skill did not trigger and you are just talking to the base model.

INSTALL INTO THE CURRENT PROJECT

```
npx skills add https://github.com/cloudflare/security-audit-skill \
  --skill security-audit
```

INSTALL ONCE FOR EVERY PROJECT

```
npx skills add https://github.com/cloudflare/security-audit-skill \
  --skill security-audit \
  --global
```

SEE THE AGENT-SELECTION AND NON-INTERACTIVE OPTIONS

```
npx skills --help
```

QUICK TIPS

- A direct audit or pen-test request gets full audit mode. A general security question gets guidance mode instead, which answers you without writing report files. If you want the files, ask for the report.
- Point it at a folder when the repo is large: `find security vulnerabilities in ./src` keeps the first run cheap while you learn what the output looks like.
- Run it on a branch with a clean working tree. You want to read the report against code you have not touched since.

The commands I actually run

Everything here is plain language, because that is how the skill triggers. Swap the bracketed parts.

FIRST FULL AUDIT, REPORTS KEPT OUTSIDE THE REPO

```
security audit this codebase, output to ~/audits/[PROJECT-NAME]/run-1
```

CHEAP FIRST PASS ON ONE FOLDER

```
find security vulnerabilities in ./[FOLDER]
```

FOCUSED RUN ON THE PARTS I TRUST LEAST

do a security review of [PATH], concentrate on authentication, the database layer and anything that passes user text into a model, output to ~/audits/[PROJECT-NAME]/focused

SECOND PASS, BECAUSE ONE RUN FINDS ABOUT HALF

run another security audit on this codebase, output to ~/audits/[PROJECT-NAME]/run-2, use the previous ledger and findings to target what was missed rather than repeating covered ground

TURN THE REPORT INTO FIXES

read REPORT.md and FINDINGS-DETAIL.md in ~/audits/[PROJECT-NAME]/run-1, then fix the confirmed findings one at a time, smallest blast radius first, and show me the diff before you apply each one

When it breaks

Everything in this table is a failure mode of this setup specifically, not generic terminal advice.

ERROR	FIX
The agent answers the security question in prose and never writes any files	It picked guidance mode. Ask for the audit directly and name the artifacts: "do a full security audit and write the report files".
No architecture.md or coverage-ledger.json appears	The skill did not trigger. Check the folder name matches the name: field in SKILL.md, and for Codex check AGENTS.md actually points at the skill folder.
Validator step fails or the agent says it cannot run the .cjs files	Node.js is missing or not on PATH. Run node --version, install Node, open a new terminal, then re-run the audit.
REPORT.md is almost empty and NEEDS-VALIDATION.md is long	No OS-enforced sandbox, so target code was never executed. Expected behaviour. Set up the sandbox, or work the needs_validation entries by hand.
Report files appear inside the repo and show up in git status	You named an output path that version control does not ignore. Move the directory outside the repo, or add it to .gitignore before the next run.
The second run repeats the same findings instead of finding new ones	Point it at the previous run's ledger and findings explicitly in the prompt. Runs are additive only when it can read the earlier output.
A finding has no severity and reads like a question	That is a needs_validation record, by design. It names the exact unresolved fact. Resolve that fact and re-run, or verify it yourself.
It flags a missing security layer you decided not to build	Cloudflare's own rule is that a defense-in-depth gap is a hardening note, not a vulnerability. If layer A already stops the attack, say so in the prompt and it will stop treating layer B as a finding.

Ready to go deeper?

This playbook is the short version. Inside the hub you get the full build walkthroughs, the prompt library, and the systems I use to ship this in a weekend.

Join the Build →

<https://hub.digicuratoragency.com/join>